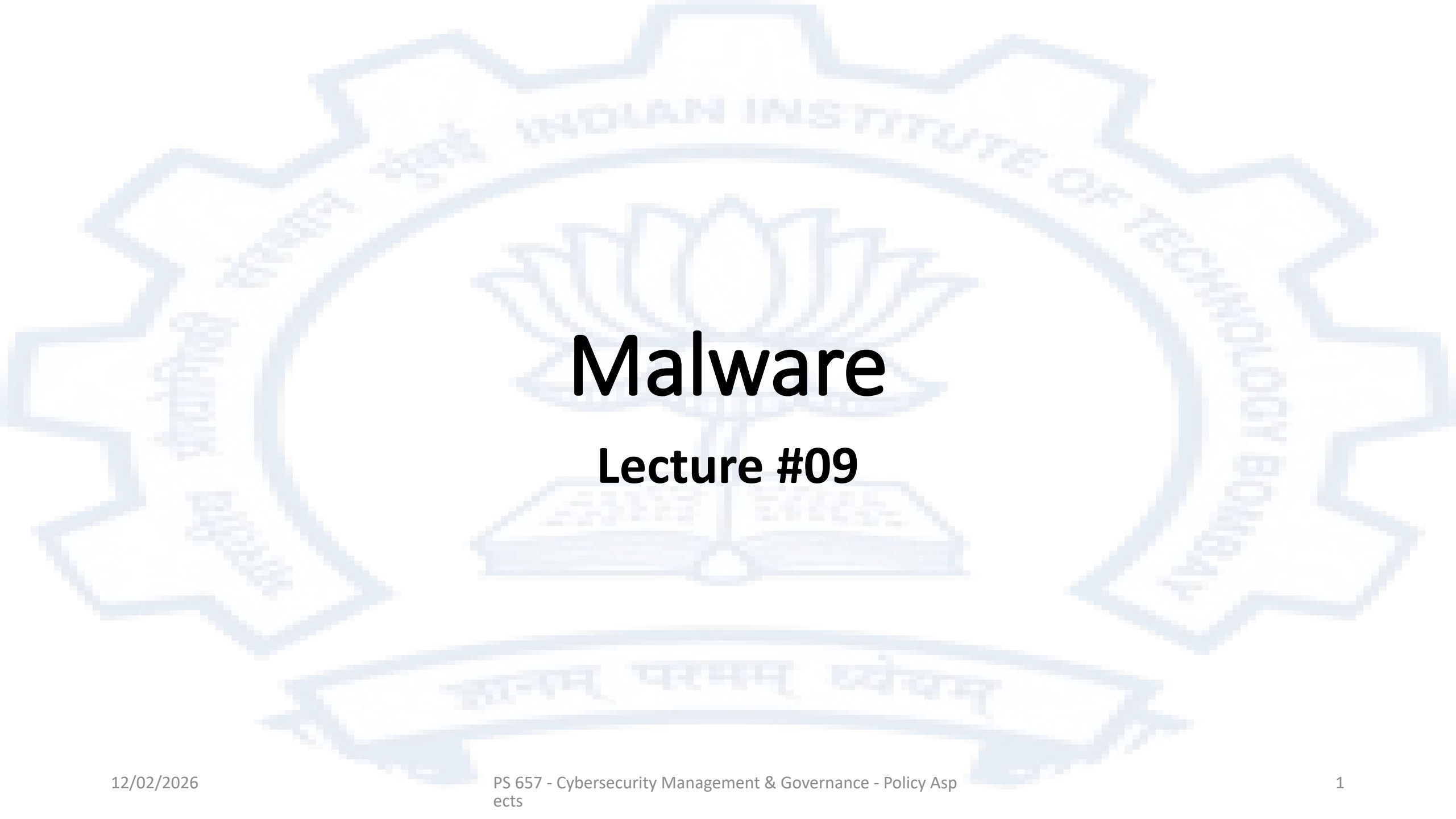


The background of the slide features a large, light blue watermark of the Indian Institute of Technology Bombay logo. The logo is circular with a gear-like outer border. Inside the border, the text "INDIAN INSTITUTE OF TECHNOLOGY BOMBAY" is written in English, and "भारतीय प्रौद्योगिकी संस्थान बॉम्बे" is written in Hindi. In the center of the logo is a stylized lotus flower above an open book.

Malware

Lecture #09

Malware

Malware or malicious software is software that is designed to perform undesirable activities such as (but not limited to) data destruction, data exfiltration, data modification, bypassing of security controls, granting access to restricted functionality, clogging network bandwidth, overusing processor capacity and many other such activities.

There are several types of malware and it is useful to have a taxonomy of malware so that classes of attacks can be understood and defended against.

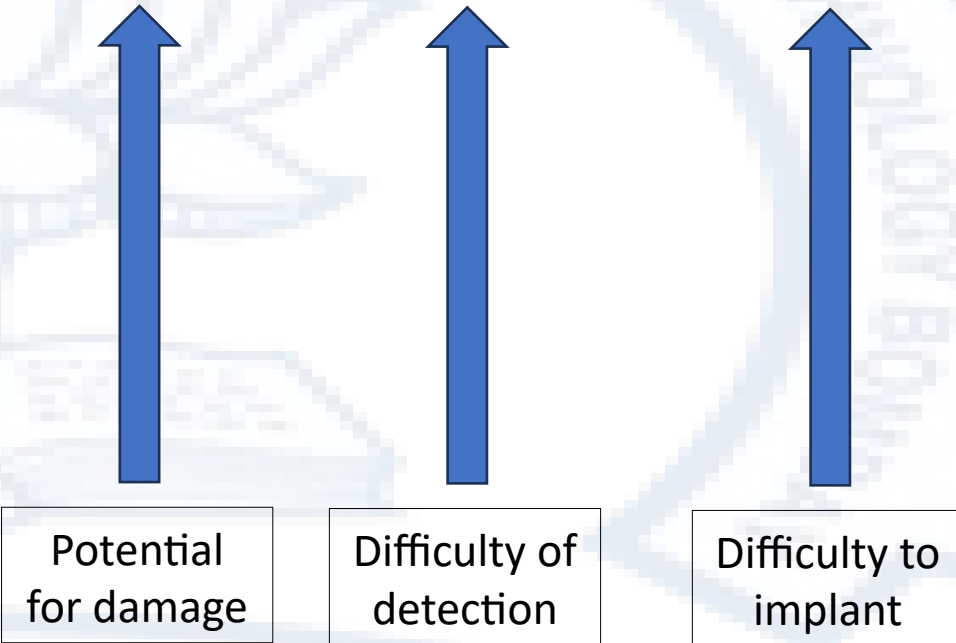
Dimensions of Malware

1. Standalone or embedded into a host program or system
 - Standalone malware is a complete program on its own. Worms and malicious botnets are examples of this type of malware.
 - Embedded malware is contained within some host program. Many viruses or injected code are examples of this type of malware
2. Persistent or transient
 - Some malware remain on the system even after it is re-booted. Such malware needs to resident on the permanent storage of a device. Such malware is called persistent malware
 - Transient malware cannot survive a reboot and is resident only in memory. Such malware can evade detection since once a system is re-booted the malware is gone and disk scans do not reveal anything

Dimensions of Malware- contd

3. Location in the software stack

- Firmware
- Boot sector
- OS kernel
- System drivers
- APIs
- User applications



Dimensions of Malware- contd

4. Auto replicating and auto running or requires user action

- Auto replicating malware does not require user action to replicate. The malware replicates via vulnerabilities in target systems or by leveraging auto run features of software stacks
- Malware that is activated by user action requires a user to carry out some action like download a file or click a link to be installed or activated. Such malware can spread automatically but requires user action on the target systems to actually produce the malicious behaviour

Dimensions of Malware- contd

5. Static or dynamic

- Static malware does not change once it has been installed
- Dynamic malware is able to upgrade itself by connecting to a back end infrastructure. One of the reasons for the need to upgrade is to evade detection which may be based on the characteristics of the old version. Another reason may be that the initial attack may only allow a limited amount of code to be injected which can then be used to bootstrap more complex functionality. Most modern malware is supported by a backend infrastructure that can update it

Dimensions of Malware- contd

5. Static or dynamic

- Static malware does not change once it has been installed
- Dynamic malware is able to upgrade itself by connecting to a back end infrastructure. One of the reasons for the need to upgrade is to evade detection which may be based on the characteristics of the old version. Another reason may be that the initial attack may only allow a limited amount of code to be injected which can then be used to bootstrap more complex functionality. Most modern malware is supported by a backend infrastructure that can update it

Dimensions of Malware- contd

6. Acts alone or acts in coordination

- Some malware acts alone and is not aware of or coordinated with other malware
- Some malware is part of large networks (botnets) that are able to carry out coordinated and large scale attacks

Using the dimensions (taxonomy) to classify specific malware

	standalone or host-program	persistent or transient	layers of system stack	auto-spreading?	dynamically updatable?	coordinated?
viruses	host-program	persistent	firmware and up	Y	Y	N
malicious browser extensions	host-program	persistent	application	N	Y	Y
botnet malware	both	persistent	kernel and up	Y	Y	Y
memory-resident malware	standalone	transient	kernel and up	Y	Y	Y

CyBOK v1.1.0

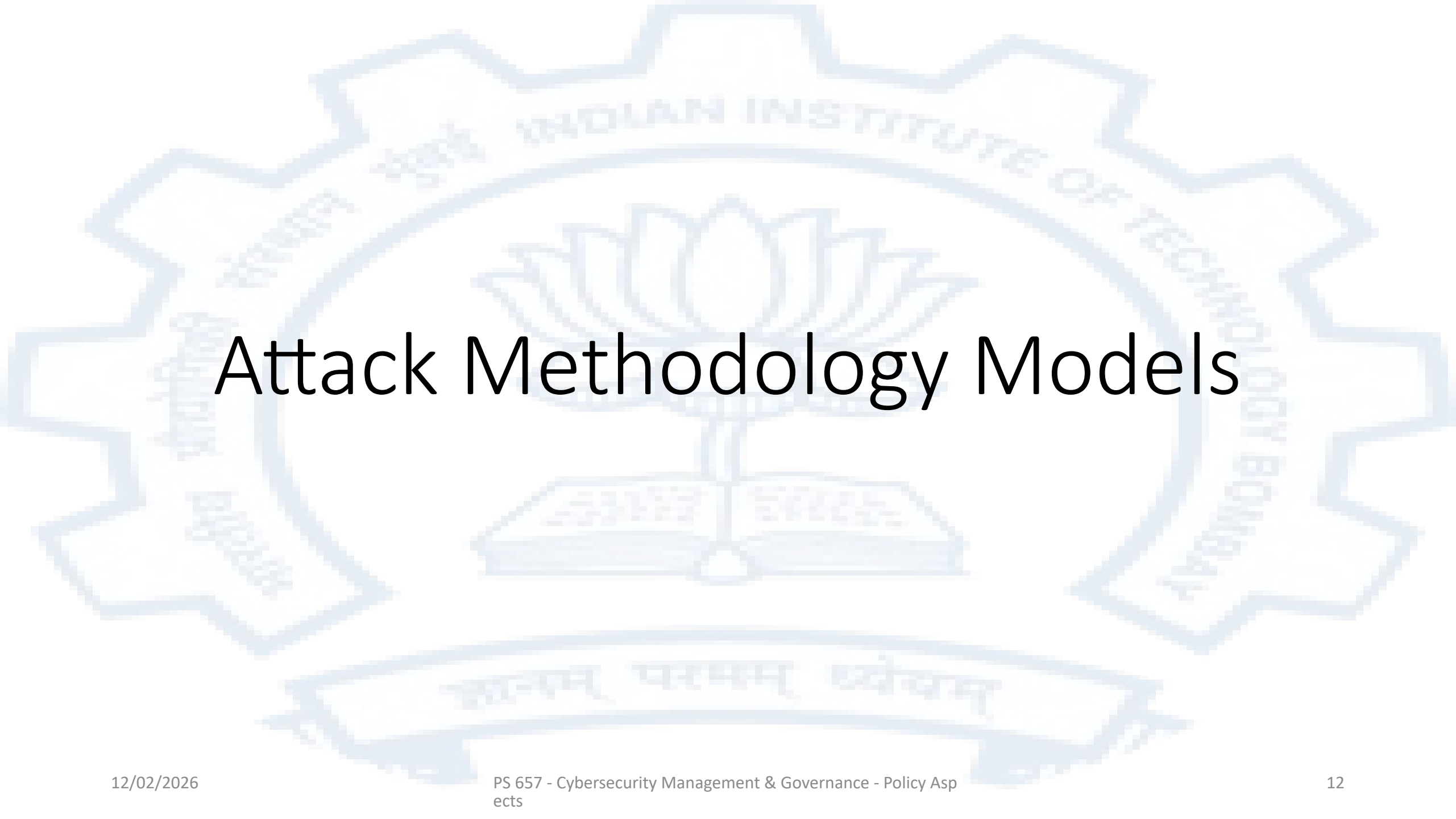
The “damage” dimension

The damage that malware can do may be

- To confidentiality of data
- To integrity of data and systems
- To availability of data and systems

Potentially unwanted programs (PUPs)

- Typically code that is a part of some useful program downloaded by a user
 - Adware in free games
- Such potentially unwanted programs may harvest a lot of sensitive and private data
- Once installed they have the ability to become malware and the user is at the mercy of the PUP operation. Therefore PUPs are often classified as a sort of malware

The background of the slide features a large, light blue watermark of the Indian Institute of Technology Bombay (IIT Bombay) logo. The logo is a gear-like shape with a lotus flower in the center and the text "INDIAN INSTITUTE OF TECHNOLOGY BOMBAY" around it.

Attack Methodology Models

The Cyber Kill-Chain Model

First proposed by Lockheed-Martin

Step	Activities
1 Reconnaissance	Harvesting email addresses, identifying vulnerable computers and accounts, etc.
2 Weaponization	Designing exploits into a deliverable payload.
3 Delivery	Delivering the exploit payload to a victim via email, Web download, etc.
4 Exploitation	Exploiting a vulnerability and executing malicious code on the victim's system.
5 Installation	Installing (additional) malware on the victim's system.
6 Command & Control	Establishing a command and control channel for attackers to remotely commandeer the victim's system.
7 Actions on Objectives	Carrying out malicious activities on the victim's system and network.

CyBOK v1.1.0

The MITRE ATT&CK Model

- Reconnaissance – 10 techniques
- Resource development – 8 techniques
- Initial access – 10 techniques
- Execution – 14 techniques
- Persistence – 20 techniques
- Privilege escalation – 14 techniques
- Defense evasion – 44 techniques
- Credential access – 17 techniques
- Discovery – 32 techniques
- Lateral movement – 9 techniques
- Collection – 17 techniques
- Command and control – 18 techniques
- Exfiltration – 9 techniques
- Impact – 14 techniques

<https://attack.mitre.org/#>

The evolution of the covert and largely underground malware ecosystem

- The early instances of malware were largely a nuisance and did not cause much damage. Malware today is much more sophisticated and is often deployed on the level of a full scale cyberwar. The damage caused is significant
- An underground and commercialized ecosystem has emerged for the entire cycle of malware which is
 - Development
 - Deployment
 - Operation
 - Monetization
- Specialized entities exist that offer these capabilities as a service

Malware analysis techniques

Malware analysis is required to understand what malware does and how it does it so that detection and mitigation is possible. The most common techniques are

- Static analysis
- Dynamic analysis
- Fuzzing
- Symbolic execution

Malware analysis environments

	Machine Emulator	Type 2 Hypervisor	Type 1 Hypervisor	Bare-metal machine
Architecture	Code-based architecture emulation	Runs in host OS, provides virtualisation service for hardware	Runs directly on system hardware	No virtualisation
Advantages	Easy to use, Fine-grained introspection, Powerful control over the system state	Easy to use, Fine-grained introspection, Powerful control over the system state	Medium transparency, Fine-grained introspection, Low overhead for hardware interaction	High transparency, No virtual environment artifacts
Disadvantages	Low transparency, Unreliability support of architecture semantics	Low transparency, Artifacts from para-virtualisation	Less control over the system state	Lack of fine-grained introspection, Scalability and cost issues, Slower to restore to clean state
Examples	Unicorn [618], QEMU [619], Bochs [620]	VirtualBox [621], KVM [622], VMware [623]	VMware ESX [624], Hyper-V [625], Xen [626]	NVMTTrace [627], BareCloud [603]

Table 6.3: Comparison of Malware Analysis Environments

CyBOK 1.1.0

Safety of Malware Analysis Environments

- Malware can be dangerous and can cause harm when being run to analyse it. Therefore safety of the analysis environment is a very important requirement
- Given the importance of reality most malware analysis environments are virtual environments. Examples are
 - The GQ system
 - The Potemkin system

Evading analysis

- Source code of Malware is rarely available therefore the first step of a malware analyst is to “disassemble” the binary code to assembly language. Malware authors use byte reuse to cause the disassembly process to produce incorrect code
- Malware authors use code obfuscation techniques to evade analysis. Some commonly used methods are
 - Packing : part of the code is compressed and encrypted
 - Control flow obfuscation
 - Code emulation: The malware author designs an abstract machine and converts the malware binaries into bytecode for that abstract machine and also includes a native code emulator for that abstract machine

Identifying analysis environments

Malware often uses fingerprinting techniques to detect the presence of an analysis environment and evade dynamic analysis (e.g., it stops executing the intended malware code). More generally, malware behaviours can be 'trigger-based' where a trigger is a run-time condition that must be true. Some examples of such triggers are

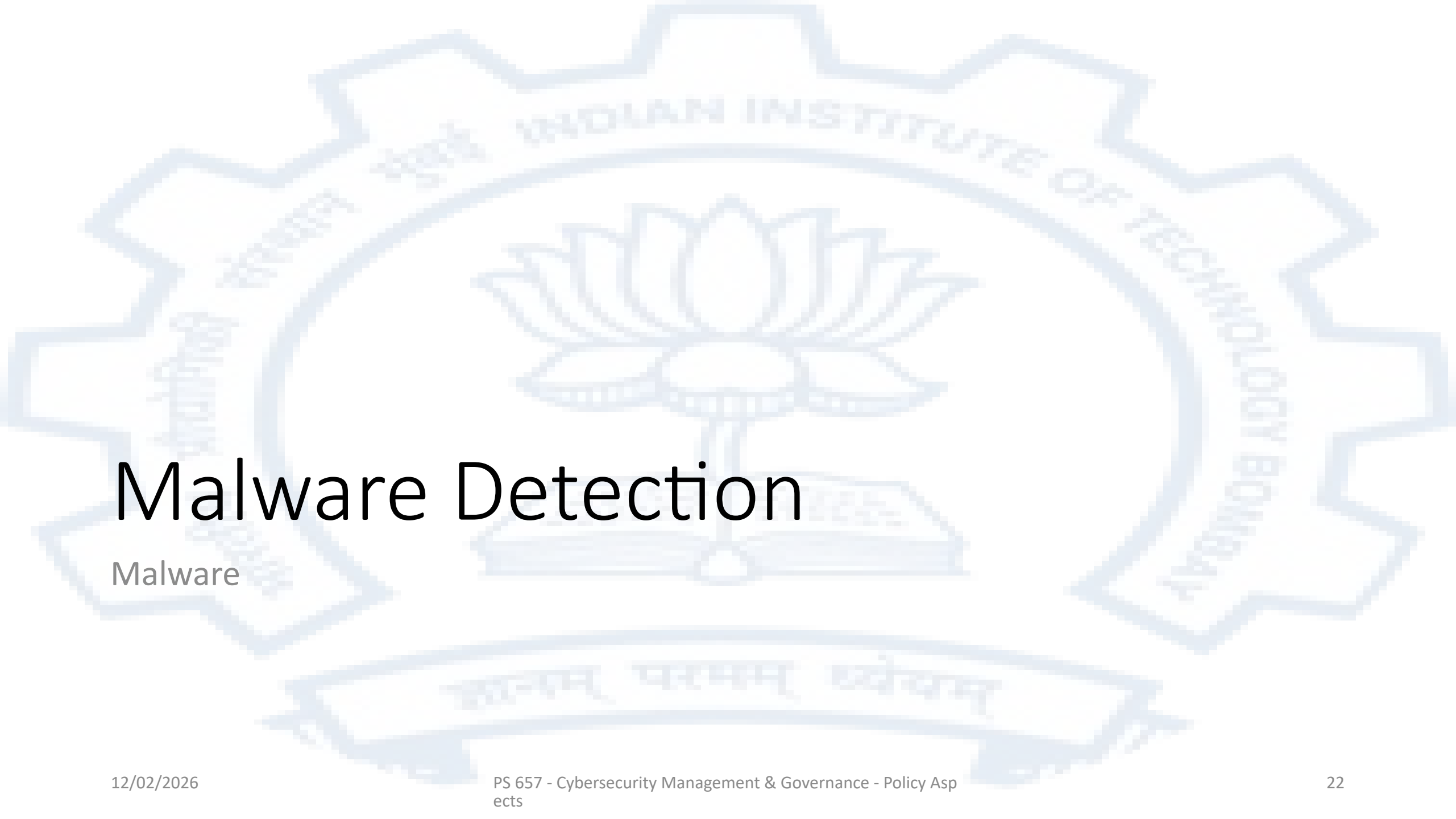
- Correct date and time
- Presence of certain files or directories
- A specific type of connection to the Internet

An analysis environment is not transparent if it can be detected by malware

Environment fingerprinting techniques

Four important type of methods are

- **Virtualisation fingerprinting** : Evasive malware tries to detect that it is running in a virtualised environment. For example, it can use red pill testing, which entails executing specific CPU instruction sequences that cause overhead, unique timing skews, and discrepancies when compared with executions on a bare-metal (i.e., non-virtualised) system.
- **Environment fingerprinting** : This relies on the fact that environment artifacts, virtual machines and emulators have unique hardware and software parameters including device models, registry values, and processes
- **Process introspection** : Malware can check for the presence of specific programs on operating systems, including monitoring tools provided by anti-virus companies and virtual machine vendors.
- **User:** User artifacts include specific applications such a web browser (or lack thereof), web browsing history, recently used files, interactive user prompts, mouse and keyboard activities etc. These are signals for whether a real human uses the environment for meaningful tasks.

The background of the slide features a large, light blue watermark of the Indian Institute of Technology Bombay logo. The logo is a gear-like shape with a lotus flower in the center. The text "INDIAN INSTITUTE OF TECHNOLOGY BOMBAY" is written along the top inner edge of the gear, and a Sanskrit motto is at the bottom.

Malware Detection

Malware

Identifying the presence of malware

- Finding malware during the distribution phase
 - Signature based scanning of network traffic, the file system and the memory of host computers
- If malware is not detected during its distribution it may still be possible to detect it when it starts executing by monitoring
 - For example many malware contact Command & Control servers over the network. Network monitoring may reveal traffic to malicious domains
- Malware authors may periodically change the malware in a way that it still has the same functionality and may similarly modify network payloads (polymorphism) in order to evade the signature and/or monitoring detection systems

Detecting malware attacks

The two broad approaches are

- Host based monitoring
- Network based monitoring

The basic idea is to monitor for anomalous behavior that would be indicative of an attack. Some examples are

- For DDoS detection, the main idea is to analyse the statistical properties of traffic, e.g., the number of requests within a short time window sent to a network server. Once a host is identified to be sending such traffic, it is considered to be participating in a DDoS attack and its traffic is blocked.
- For ransomware detection, the main approaches include monitoring host activities involved in encryption. If there is a process making a large number of *significant* modifications to a large number of files, this is indicative of a ransomware attack

Machine learning based security analytics

- ML has been applied to build models for malware and attack detection since the 90s
- ML has been quite successful in identifying botnets by classifying domain names as being generated by Domain Generation Algorithms
- The drawback with (classical) ML applied to security is that we must select or even engineer features that are useful in classifying benign and malicious activities. Feature engineering is very knowledge- and labour- intensive and is the bottleneck in applying ML to any problem domain.

Application of deep learning to security

Deep learning has shown some promise in learning from a large amount of data without much feature engineering, and already has great success in applications such as image classification. However, unlike many classical ML models (such as decision trees and inductive rules) that are human-readable, and hence reviewable by security analysts before making deployment decisions, deep learning outputs blackbox models that are not readable and not easily explainable. It is often not possible to understand what features are being used (and how) to arrive at a classification decision. That is, with deep learning, security analysts can no longer check if the output even makes sense from the point-of-view of domain or expert knowledge.

Malware response – infected assets are accessible

- Remove the malware
- Recover data from secure backups
- Restore services
- Add new signatures to IDS systems
- Add new firewall rules

Malware takedowns

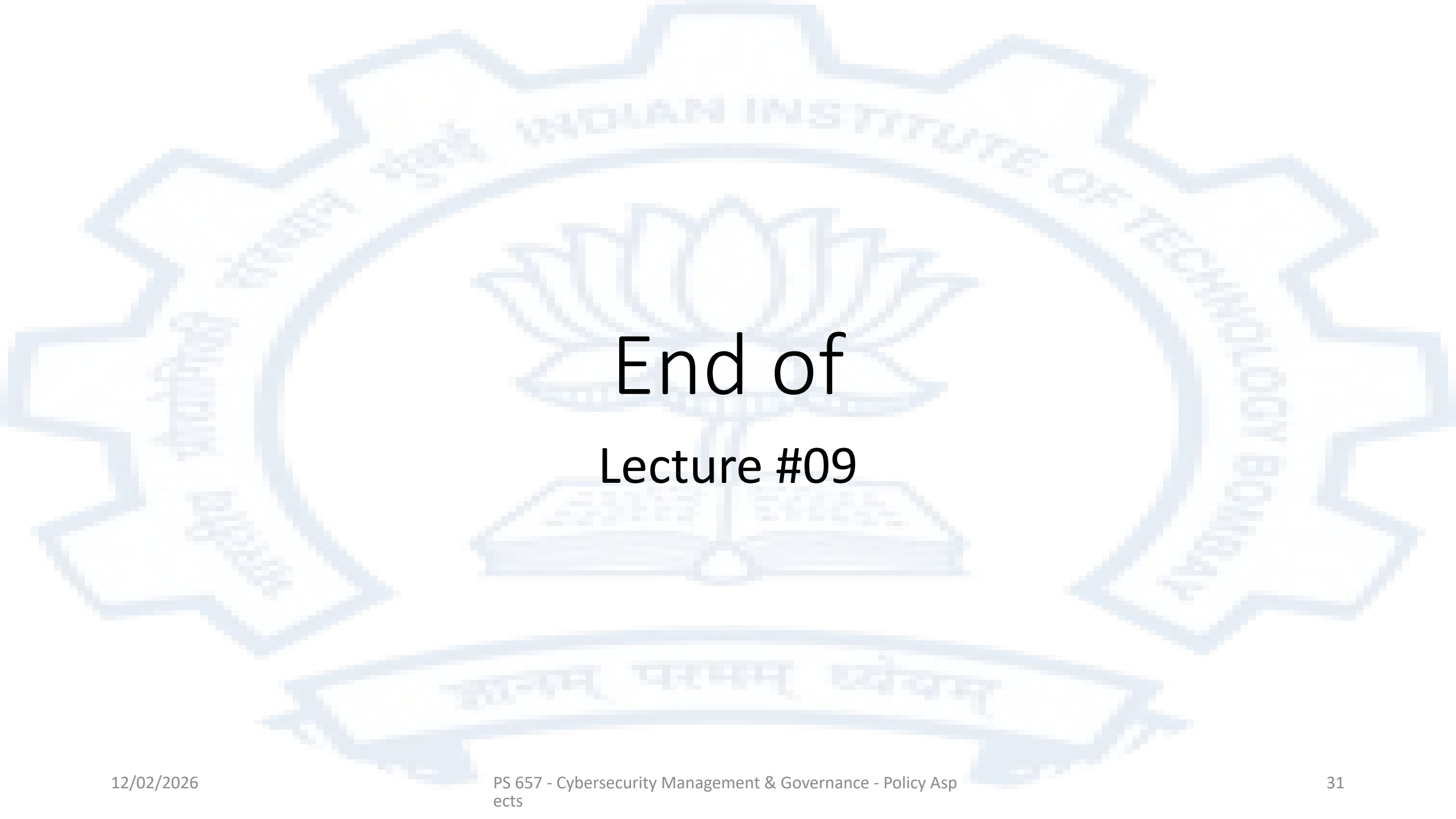
- If the infected assets are not easily accessible or the attack is at a large scale then the malware operation may have to be disrupted or taken down
- There are several types of takedowns to disrupt malware operations. If the malware uses domain names to look up and to communicate with centralised C&C servers, we perform takedown of C&C domains by 'sinkholing' the domains, i.e., making the C&C domains resolve to the defender's servers so that botnet traffic is 'trapped' (that is, redirected) to these servers. If the malware uses peer-to-peer (P2P) protocol as a decentralised C&C mechanism, we can partition the P2P botnet into isolated sub-networks, create a sinkholing node, or poison the communication channel by issuing commands to stop the malicious activities. However, it should be borne in mind that, in most territories active defence or intelligence gathering, such as hack-backs, access to or modification of servers, DNS, or networks, is unlawful without appropriate legal authority.

Attribution

- Ideally, law enforcement wants to identify the actual criminal behind the attacks. Identifying the virtual attacker is an important first step toward this goal. An attacker may have consistent coding styles, reuse the same resources or infrastructures, or use similar C&C practices.
- From the malware data, we can compare its 'characteristics' with those of known historical adversaries, e.g., coding styles, server configurations, etc. At the source code level, we can use features that reflect programming styles and code quality.
- From the enumerated attack infrastructure, we can associate the expanded domain name set with previously known adversaries.

To summarize

- Attackers use malware to carry out malicious activities on their behalf.
- Malware can reside in any layer of the system stack, and can be a program by itself or embedded in another application or document.
- Modern malware comes with a support infrastructure for coordinated attacks and automated updates, and can operate low-and-slow and cover its tracks to avoid detection and attribution
- While malware can cause wide-spread infection and harm on the Internet, it can also be customised for attacks targeting a specific organization
- Malware analysis is an important step in understanding malicious behaviours and properly updating our attack prevention and detection systems.
- Malware employs a wide range of evasion techniques, which include detecting the analysis environment, obfuscating malicious code, using trigger-conditions to execute, and applying polymorphism to attack payloads, etc. Accordingly, we need to make analysis environments transparent to malware, continue to develop specialised program analysis algorithms and machine-learning based detection techniques, and apply a combination of these approaches.
- Response to malware attacks goes beyond detection and mitigation, and can include take-down and attribution

The background of the slide features a large, light blue watermark of the Indian Institute of Technology Bombay (IIT Bombay) logo. The logo is a gear-like shape with a lotus flower in the center. The text "INDIAN INSTITUTE OF TECHNOLOGY BOMBAY" is written along the top inner edge of the gear, and "सत्यमेव जयते" is written along the bottom inner edge.

End of Lecture #09